

From 2025年11月3日

📁 算法新知与思路 (New Ideas)

📊 图论 (Graph)

- **P4017 最大食物链计数**: 拓扑排序 DFS+DP DAG计数
- **P4779 单源最短路径**: Dijkstra 优先队列优化 标准模板
- **图的遍历 / Tarjan**: Tarjan缩点 反向建图 记忆化DFS DAG剪枝

📐 计算几何 (Geometry)

- **牛牛战队的秀场**: 正多边形内接 几何公式
- **牛牛战队的比赛地 (三分教学)**: 三分法 凸函数极值 二分答案对比

🎯 贪心与思维 (Greedy & Thinking)

- **Cool Partition**: Set去重 区间划分
- **C. Range Operation**: 前缀和优化 公式变形 分离变量 线性扫描
- **C. Monopati (优先队列优化)**: 优先队列 固定左端点扫右端点 区间覆盖
- **P2887 Sunscreen**: 区间贪心 排序策略
- **P2949 Work Scheduling G**: 反悔贪心 大根堆 带期限任务
- **C. Cyclic Merging**: 环形处理 链表模拟 合并代价

🏠 动态规划 (DP)

- **DP 核心心法**: 状态定义 滚动数组 无后效性
- **P1115 最大子段和**: 线性DP 经典模型
- **A. Against the Difference**: 子序列DP 打表辅助 桶优化
- **P4310 绝世好题**: 位运算DP 按位拆分 $O(n \log n)$ 优化
- **最长上升子序列 (LIS)**: Dilworth定理 二分优化 $O(n \log n)$

🎲 博弈论 (Game Theory)

- **Enjoy the game**: 奇偶性博弈 位运算判断 必胜策略

🔗 数据结构与容器

- **CF Div1+2 C (Set/Map)**: Multiset lower_bound 动态维护有序集合 贪心匹配

⚠️ 错题警示录 (Mistakes)

- **复杂度与容器选择 - P2058 海港**: TLE警示 队列模拟 桶计数
- **读题与边界 - 签到题**: 读题失误 Set误用
- **二维前缀和边界 - P2004 领地选择**: 滑动窗口 下标越界 推导公式
- **双指针细节 - 子段乘积**: 滑动窗口 取模逆元 1-Based下标

✂ 实战复盘 (Contests)

- **11.2 华工新生赛:**
 - **G 因数分解:** 博弈 控制权压缩
 - **D 水群:** Dijkstra 有向图最短路
 - **E 代号N:** 带权无向图 优先队列 BFS/DFS路径
- **11.2 牛客挑战赛 - 小红的区间构造:** 差分数组 构造 前缀和
- **11.5 牛客基础组 - 子段异或和:** 异或前缀和 Map优化 子段计数
- **GZHU 2024 VP - ACM中的ACM:** Map套Map 模拟 大模拟心态
- **数论 - 逆=i+英:** 同余 桶计数 复杂度分析

📁 附件

- **图论名词速查表**

新题新思路

大概是我绞劲脑汁都没想到如此精妙的解法

图论

P4017 最大食物链计数 (拓扑排序, 挺简单的)

- **题目概述** 给出一张有向无环图, 求出最长路径的数量 (最长路径定义: 入度为0的点到初读为0的点), n是节点数量, m是路径数量
- **数据范围:** n:2e3, m:1e5
- **初始思路;** 记忆化加DFS
- **正解思路:** DFS+DP
- **AC代码**

```
int mod = 80112002;
int n, m;
vi mem(5005, -1);
int dfs(int u, const vector<vi> &dw, const vi&in, const vi&out)
{
    if (mem[u] != -1)
    {
        return mem[u];
    }
    if (in[u] == 0)
    {
        mem[u] = 1;
        return 1;
    }
    mem[u]=0;
    for (int v : dw[u])
    {
        // mem[u] = max(mem[u], dfs(v, dw,in,out) % mod + 1);
        mem[u]=(dfs(v,dw,in,out)+mem[u])%mod;
    }
}
```

```

    }
    return mem[u];
}

void solve()
{
    vector<vi> dw(5005);
    cin >> n >> m;
    vi out(n + 1);
    vi in(n + 1);
    rep(i, 0, m - 1)
    {
        int d, f;
        cin >> d >> f;
        dw[f].push_back(d);
        out[d]++; // 需要计算入度和出度是为了辨别什么时候开始什么时候结束
        in[f]++;
    }
    int ans = 0;
    for (int i = 1; i <= n; i++)
    {
        if (out[i] == 0)
            // ans = max(ans, dfs(i, dw) % mod);
            ans = (ans + dfs(i, dw, in, out)) % mod;
    }
    cout << ans << '\n';
}

```

• AC代码:拓扑排序

```

void solve()
{
    vector<vi> dw(5005);
    cin >> n >> m;
    vi out(n + 1);
    vi in(n + 1);
    rep(i, 0, m - 1)
    {
        int d, f;
        cin >> d >> f;
        dw[f].push_back(d);
        out[f]++;
        in[d]++;
    }
    int ans = 0;
    queue<int> dl;
    vi lx(n + 1, 0);
    for (int i = 1; i <= n; i++)
    {
        if (in[i] == 0)
        {
            dl.emplace(i);

```

```

        lx[i]=1;
    }
}

while (!dl.empty())
{
    auto hsh = dl.front();
    dl.pop();

    for (auto hsh2 : dw[hsh])
    {
        lx[hsh2]=(lx[hsh2]+lx[hsh])%mod;
        in[hsh2]--;
        if(in[hsh2]==0)
        {
            dl.emplace(hsh2);
        }
    }
}

for(int i=1;i<=n;i++)
{
    if(!out[i])
    {
        ans=(ans+lx[i])%mod;
    }
}

cout << ans << '\n';
}

```

- 拓扑排序的目标是将所有节点排序，使得排在前面的节点不能依赖于排在后面的节点。
- 作用：
 - 确定任务执行顺序
 - DAG 上的动态规划
 - 检测环路:如果拓扑排序无法将所有节点都加入到最终的序列中（
 - **“顺序”、“依赖”、“先决条件”，或者需要在一个有向图中进行基于依赖的计算（如 DP）时

简单 Dijkstra 模板题

P4779 【模板】单源最短路径（标准版）

- 题号: P4799
- 链接: [题目链接](#)
- 算法类型: 图论模板
- AC 代码:

```

void solve()
{
    ll n,m,s;
    cin>>n>>m>>s;
    //前面是点,后面是距离

```

```

vector<vector<pair<int,ll>>> mp(n+1);
rep(i,0,m-1)
{
    int u,v,w;
    cin>>u>>v>>w;
    mp[u].push_back({v,w});
    // mp[v].push_back({u,w});这道题是有向边所以不要入两次!!
}

priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>>pos ;
vector<ll> dist(n+1,LINF);
dist[s]=0;
pos.emplace(0,s);

while(!pos.empty())
{
    auto [nowd,nowp] = pos.top();
    pos.pop();
    if(nowd>dist[nowp])continue;
    for(auto [nextp,nextd]:mp[nowp])
    {
        if(dist[nextp]>dist[nowp]+nextd)
        {
            dist[nextp]=dist[nowp]+nextd;
            pos.emplace(dist[nextp],nextp);
        }
    }
}
rep(i,1,n)
{
    cout<<dist[i]<<' ';
}
}

```

- **注意事项:**
 - 注意Dijkstra算法用最小堆优化可以时间复杂度最低
 - Dijkstra算法只能处理非负权路径问题
 - 为什么不用队列? : 贪心最快, 如果是菊花图复杂度会退化到nm
 - 含负权路用什么算法? 用队列
- **思路:**
 - 认真研读并学习114514次

图的遍历 (tarjan算法)

- **题目描述:** 给出一张有向图 (不保证无环), 节点编号1到n, 求每个节点能到达的最大编号
- **错误解法: 我一开始想到的:** dfs加记忆化, 从入度为0的点开始dfs到出度为0的点, 每个点的答案在确认的时候和自己还有其他的路径比较一下;
 - **错解中的逻辑问题**

- 只D入度为0的点，但是当图中出现环就会漏掉
- 为了剪枝设计了vis数组来标记有没有被访问过，但是没有重置：这个题目不保证无环，所以必须重置，甚至说这个vis的存在就没啥必要
- 在有环图中，当DFS访问到一个正在递归栈中的点时，说明遇到了环。正确的DFS应该使用三态
- 在ans中要先初始化ans[i]=i，因为每个点都可以到达自己。**以免出现没有连通的点**

vis在以下情况不需要重置：当你的图是有向无环图 (DAG) 或者你对每个点的计算结果是确定的、最终的、且不会随起点变化时，你可以设置 vis 后不再变回 1 当他作为强连通分量 (SCC) 标记也可以不重置。比如说在这道题里面，当你反向建图让他从最大数字的点开始DFS的时候vis就可以选择不重置：**因为N在这条路径上是最大的点，后面的状态可以通过继承前面的状态来确定答案**

- **正解一：反向建图**：原理：贪心保证剪枝成功(一旦一个点的答案 $A(v)$ 被确定，它就是正确的最大值，且永远不需要重新计算。)
 - 问题: 为什么 Visited(v) 可以永不重置?
 - 思考: 当我们从 i 开始 DFS 时，如果 v 还没有 Visited，我们就设 $A(v) = i$ 。在此之前，所有的 $k > i$ 都没有在 G' 中到达 v (否则 v 早就被标记了)。因此，没有比 i 更大的点是 v 可达的。
- **正解二：tarjan算法**：求“缩点”操作的高效算法
 - 缩点然后顺序dfs+dp。求完强连通分量可以保证图片是DAG
- **检查自己方案合理性的思考路径**：
 - 1. 图的特性：有向？无环？连通性？（有没有孤立的点）边权？（负权边？）
 - 2. 做法检验：有没有环？依赖顺序是什么？状态是否能持久？（影响剪枝）（在 DAG 上，从拓扑序逆序（即从终点开始）计算是可靠的。原图 DFS 依赖于子问题的答案，必须确保子问题先被计算。）
- **AC正解1(反向图思路)**

```
void dfs(int a, int f, const vector<vi> &mp, vi &ans)
{
    ans[a] = f;
    if (mp[a].empty())
        return;
    for (int hsh : mp[a])
    {
        if(ans[hsh] != -1)
        {
            continue;
        }
        dfs(hsh, f, mp, ans);
    }
    return;
}

void solve()
{
    int n, m;
```

```

cin >> n >> m;
vector<vi> mp(n + 1);
rep(i, 1, m)
{
    int u, v;
    cin >> u >> v;
    mp[v].push_back(u);
}
vi ans(n + 1, -1);
for(int i=n;i>=1;i--)
{
    if(ans[i]!=-1)
    {
        continue;
    }

    dfs(i,i,mp,ans);
}
rep(i,1,n)
{
    cout<<ans[i]<<' ';
}
}

```

- **AC正解2**(Tarjan思路)

数据结构

Cool Partition

- **题号:** Div3C
- **算法类型:** 贪心
- **AC 代码:**

```

void solve()
{
    int n;
    cin >> n;
    set<int> basedonen;
    set<int> basedln;
    int ans = 0;
    rep(i, 1, n)
    {
        int d;
        cin >> d;
    }
}

```

```

        basedln.emplace(d);
        basedonen.emplace(d);
        if (basedln.size() == basedonen.size())
        {
            ans++;
            basedln.clear();
        }
    }
    cout << ans << '\n';
}

```

• 注题目解析

- 事实上这是一个set使用快乐题，一道普及-的贪心。
- 需要划分区间尽可能多地多，就是需要每个区间尽可能地小。对于一个区间存在的数的种类，必然等于从0到他的数的种类

DP

• 我该怎么看出这是一道DP

- 求最大/小值求方案数量求子序列
- 贪心做不了(有反例)
- 依赖前面的值，无后效性

• 怎么写DP

- 状态是谁？状态有哪些？状态必须包含所有能够影响下一阶段决策的信息
- 怎么转移？dp[i]是从哪里转移过来的？怎么转移代价最小/收益最大/能计算全部路径？
- 初始状态

• DP数组怎么写

- 一般求什么什么就是DP数组
- 要不要开二维？就问自己：“如果有两个人同时走到第 i 步，但他们之前的经历不同，这种不同会限制他们接下来的选择吗？”
- 能不能压缩？用滚动数组来代替二维，我们只考虑会影响结果的值。只记我们需要记的，过期的扔掉。

状态 背包问题：放或者不放；线性dp：选择或者不选择(甚至说有几个状态)；爬楼梯：走1234步？；LCS：两个指针；取石子：左右区间 要不要开二维：爬楼梯：如果大家都走到 i 层，接下来都是面对一样权重的选择，一维数组就行 01背包：考虑到第 i 个物品，如果前面拿了很多东西，我背包可能就放不下了，如果前面拿的东西不是很多我背包就放得下了，所以得开二维 01背包一维优化：原始方程： $dp[i][w] = \max(dp[i-1][w], dp[i-1][w - \text{weight}[i]] + \text{value}[i])$ （倒序遍历）：如果把二维压成一维 $dp[w]$ 。当我们计算 $dp[w]$ 时，我们希望用到的是上一轮 ($i-1$) 的旧数据。如果我们正序遍历 ($0 \rightarrow W$)：当你算 $dp[10]$ 时，可能会用到 $dp[5]$ 。但因为你是正序， $dp[5]$ 已经在这一轮被更新成第 i 轮的新数据了。这就等于你用了第 i 个物品两次（变成了完全背包）。如果我们倒序遍历 ($W \rightarrow 0$)：当你算 $dp[10]$ 时，用到的 $dp[5]$ 还没被这一轮访问过，所以它依然是第 $i-1$ 轮的旧数据。完美！

最大子段和

P1115 最大子段和

题目描述

给一个长度为 n 的序列 a ，选出其中连续且非空的一段使得这段和最大。

输入格式

第一行是一个整数，表示序列的长度 n 。第二行有 n 个整数，第 i 个整数表示序列的第 i 个数字 a_i 。

输出格式

输出一行一个整数表示答案。

数据规模与约定

- 对于 40 的数据，保证 $n \leq 2 \times 10^3$ 。
- 对于 100 的数据，保证 $1 \leq n \leq 2 \times 10^5$ ， $-10^4 \leq a_i \leq 10^4$ 。

• AC代码

```
void solve()
{
    int n;
    cin >> n;
    vi nums(n+1);

    for (int i=1; i<=n; i++) cin >> nums[i];
    vi dp(n+1);
    int ans=-4545556565;
    for(int j=1; j<=n; j++)
    {
        dp[j]=dp[j-1]+nums[j];
        if (dp[j]<=0) dp[j]=nums[j];
        ans=max(ans, dp[j]);
    }
    cout << ans << '\n';
}
```

A. Against the Difference

• 题目

We define that a block is an array where all elements in it are equal to the length of the array. For example, $[3, 3, 3]$, $[1]$, and $[4, 4, 4, 4]$ are blocks, while $[1, 1, 1]$ and $[2, 3, 3]$ are not. An array is called neat if it can be obtained by the concatenation of an arbitrary number of blocks (possibly zero). Note that an empty array is always neat. You are given an array a consisting of n integers. Find the length of its longest neat subsequence*. *A sequence c is a subsequence of a sequence a if c can be obtained from a by the deletion of several (possibly, zero or all) element from arbitrary positions. 我们定义block

是一个数组，其中的所有元素都等于该数组的长度。例如， $[3, 3, 3]$ ， $[1]$ 和 $[4, 4, 4, 4]$ 是块，而 $[1, 1, 1]$ 和 $[2, 3, 3]$ 不是块。如果数组可以通过任意数量的块（可能为零）的连接获得，则称为整齐数组。注意，空数组总是整洁的。给定一个由 n 个整数组成的数组 a 。求它的最长整洁子序列 * 的长度。* 序列 c 是序列 a 的子序列，如果 c 可以从 a 中删除任意位置的几个（可能为零或全部）元素。

• 思路

- 首先这道题目是典型dp，因为他是通过删掉一些数字找一个最大的子序列，对于每个数字来说他面临两种状态:删掉或者不删掉(或者说是选择或者不选择)。而决定他是否被选择的权值就是到目前这个位置他做多少做贡献。容易发现只有在他结算的时候才会做贡献。我们只需要贪心地(我的意思是他要找离他最近地跟他数字相同的，这样子才能给后面留空间。因为递推所以有 $i < j$ 必然有 $dp[i] \leq dp[j]$)找他开头的那一时刻的那一时刻结算就行。

• AC代码：用桶和打表数组来处理每个数字的位置

```
void solve()
{
    int n;
    cin >> n;
    vi nums(n+1);
    vector<vi> idx(n+1);
    vi ery(n+1);
    rep(i, 1, n)
    {
        cin >> nums[i];
        idx[nums[i]].push_back(i);
        ery[i] = idx[nums[i]].size();
    }

    vi dp(n+1);
    dp[0] = 0;
    vi xz(n+1);
    for(int i = 1; i <= n; i++)
    {
        int pans = 0;
        int cur = ery[i] - nums[i];
        if(cur >= xz[i] * nums[i])
        {
            pans = dp[idx[nums[i]][cur] - 1] + nums[i];
            xz[nums[i]]++;
        }
        dp[i] = max(pans, dp[i-1]);
    }

    cout << dp[n] << '\n';
}
```

P4310 绝世好题（位运算）

不辜负他的题目名字，确实是绝世好题

• 题目

给定一个长度为 n 的数列 a_i ，求 a_i 的子序列 b_i 的最长长度 k ，满足 $b_i \& b_{i-1} \neq 0$ ，其中 $2 \leq i \leq k$ ， $\&$ 表示位运算取与。

- **解析：**有脑子的人一眼就能写出dp方程，解法类似于求LIS，但是这是n方做法，本题数据范围1e5。必然要想出一种nlogn级别的优化，怎么优化呢？实现上这是一种位运算特有的思路：每一个位置拆开来进dp

• AC代码

```
vi dp(32,0);
int zuida=-1;
for (int hsh : nums)
{
    for (int i = 0; i < 32; i++)
    {
        if ((hsh >> i) & 1) {
            zuida=max(dp[i]+1,zuida);
        }
    }
    for (int i = 0; i < 32; i++)
    {
        if ((hsh >> i) & 1) {
            dp[i]=zuida;
        }
    }
}

cout<<zuida;
```

计算几何

牛牛战队的秀场

- **题目简述：**给定圆半径r，求里面内接正n边形边长
- **代码块**

```
#define M_PI 3.14159265358979323846
void solve()
{
    int n;
    double R;
    cin >> n >> R;
    double side = 2 * R * sin(M_PI / n);
```

```

int q,w;
cin>>q>>w;
int jl=min(abs(w-q),n-abs(w-q));
cout << fixed << setprecision(10) << side*jl<< '\n';
}

```

- **公式:** `double side = 2 * R * sin(M_PI / n);`

牛牛战队的比赛地(三分教学)

- [link](#)
- **题意简述:** 给出若干个点的坐标, 在x轴上找到某点使得该点与给出的点距离的最大值最小, 求这个最大值最小的值(数据范围:点的数量 $1e5$, 坐标范围 $-1e9$ 到 $1e9$)
- **tag:** 计算几何, 二分答案/三分
- **思路做法:**
 - 首先读题: 距离的最大值最小值, 再看数据范围: 遍历法: $1e23$??
 - 问题转化: 这里我们有两种转化思路: 1.找到最大值最小的值, 对可能的答案进行二分。求: 是否有一个答案满足: 对于所有点来说, 必然存在一个共同的点(或者区间)在x轴上, 与他距离为r -> 求圆覆盖交集问题 -> cek函数: 求交集 思路2.按照题目的顺思路: 对x轴进行三分, 找到符合要求的点, 不需要写cek, 直观简单 缺点: 我不会三分
- **新算法: 三分**
 - 和二分的区别: 二分对象具有单调性;判断“是否可行” → 二分答案(如最小化最大值) 三分直接求凸函数极值(无 check 可写)
 - 三分用处: 适合答案有类似于凸函数的性质
 - 求函数最值
 - 求唯一最小值
 - 求最远点对, 圆覆盖问题转化
 - 概率期望优化
 - 注意事项:
 - 在 $1e9$ 到 $1e12$ 循环100次
 - 函数需要单峰
 - 整数域使用最好用二分
 - 三分模板

```

double ternary_search(double l, double r) {
    for (int i = 0; i < 100; i++) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        if (f(m1) < f(m2)) r = m2;
        else l = m1;
    }
}

```

```

    }
    return (1 + r) / 2;
}

```

- **AC代码：二分答案（适合答案有单调性）**

```

struct node
{
    double x, y;
};
bool check(double r, const vector<node> &jd)
{
    double le = -1e9;
    double ri = 1e9;
    for (node hsh : jd)
    {
        if (fabs(hsh.y) > r + EPS)
            return false; //剪枝
        double dy = hsh.y;
        double dx = sqrt(max(0.0, r * r - dy * dy));
        le = max(le, hsh.x - dx);
        ri = min(ri, hsh.x + dx);
    }
    return le <= ri + EPS;
}
void solve()
{
    int n;
    cin >> n;
    vector<node> jd(n);

    rep(i, 0, n - 1)
    {
        double a, s;
        cin >> a >> s;
        jd[i].x = a;
        jd[i].y = s;
    }
    double l = 0, r = 3e4;
    for(int i=0;i<=100;i++)
    {
        double mid = (l + r) / 2;
        if (check(mid,jd))
            r = mid;
        else
            l = mid;
    }
    cout << fixed << setprecision(10) << l << '\n';
}

```

- **AC代码**：三分（适合答案有凸函数的性质）

```

struct Point
{
    double x, y;
    Point(double x = 0, double y = 0) : x(x), y(y) {}
    Point operator-(const Point &b) const { return Point(x - b.x, y - b.y); }
    double len() const { return hypot(x, y); }
};

double Dist(Point a, Point b) { return (a - b).len(); } // 需要重载.len()
double mxdist(const vector<Point> &rec, Point c)
{
    double s = 0;
    for (Point hsh : rec)
    {
        s = max(Dist(hsh, c), s);
    }
    return s;
}

void solve()
{
    int n;
    cin >> n;
    vector<Point> rec(n);
    rep(i, 0, n - 1)
    {
        cin >> rec[i].x >> rec[i].y;
    }
    double l = -1e9;
    double r = 1e9;
    for (int i = 0; i < 100; i++)
    {
        double m1 = l + (r - l) / 3;
        double m2 = l + (r - l) * 2 / 3;
        Point c1 = {m1, 0};
        Point c2 = {m2, 0};
        if (mxdist(rec, c1) < mxdist(rec, c2))
            r = m2;
        else
            l = m1;
    }
    cout << fixed << setprecision(10) << mxdist(rec, {1, 0});
}

```

- **注意事项**：double有精度丢失：用 for(int i=0;i<=100;i++)实现各种操作

博弈

Enjoy the game

- [link](#)

- **题目描述**: 有 n 张牌, 轮流取牌: 第一个人最多拿 $n-1$ 张, 最少拿1张。接下来每回合最多拿上一个人拿的数量, 最少一张。拿走最后一张牌就胜利。求先手必胜策略?
- **思路分析**: 容易知道: 对于每个人来说, 到自己回合得时候能使牌堆里面的牌变成单数, 或者把这堆牌变成偶数丢给对方就必胜。对于先手来说, 他要尽可能把牌变成偶数且尽可能通过减一做到。容易知道所有奇数都会胜利。对于偶数情况, 先手第一次行动只能拿偶数牌, 要不然对面直接拿1就废了。当先手将牌堆维护在偶数时, 对面只能也拿偶数, 因为拿了奇数丢过来的还是奇数先手就赢了就不是最优策略了。所以: 如果先手能够取偶数且取偶数次(在取得次数上也限制后手)先手有必胜策略。其他情况下后手可以通过调整取偶数的次数来限制(因为双方只能拿 $2x$ 张牌, 但是对于 x 没办法保证最优解)接下来可以尝试小样例看看有没有需要特判的, 发现没有
- **代码实现**: 所以事实上我们要做的是求出 $\log_2 n$ 是否是奇数, 用while循环固然只管但是可能吃一发20分钟的罚时
- **求一个数是否是2的正整数次幂**: $(n \& (n - 1))$

trick(Educational)

C. Range Operation

• 题目

You are given an integer array a of length n . You can perform the following operation: choose a range $[l, r]$ ($1 \leq l \leq r \leq n$) and replace the value of elements $a_l, a_{l+1}, \dots, a_r$ with $(l + r)$. Your task is to calculate the maximum possible total array sum if you can perform the aforementioned operation at most once. 您将得到一个长度为 n 的整数数组 a 。您可以执行以下操作: 选择范围 $[l, r]$ ($1 \leq l \leq r \leq n$), 并将元素 $a_l, a_{l+1}, \dots, a_r$ 的值替换为 $(l + r)$ 。您的任务是计算最大可能的阵列总和, 前提是您最多可以执行一次上述操作。

• 思路

- **区间总和**->前缀和优化
- 对于每个数字我们是否能判断他更改的收益从而确定他要不要被更改, 随后贪心解决一切?
- 第一条思路: 让他和他自己比较的贪心: 但是你会发现这只能确定一些“必须更改”的数字, 不符合题目里面的边界, l 和 r 的改变随时能够影响更改的权重。这个思路假的很明显
- 第二条思路: 要去找 l 和 r 的最优值肯定是 n 方解法, 我们能不能通过假设存在 l 和 r 然后推导出对于每一组 l 和 r 的收益表达式来找到他贪心的点呢?
 - 对于确定的 l 和 r , 更改之后的收益是: $(l+r) \cdot (r-l+1) - (qzh[r] - qzh[l-1])$
 - 学过高中数学我们就知道 l 和 r 是可以分离开的: 得到贪心最大化式子: $l^2 - r^2 + l + r + qzh[r] - qzh[l-1]$
 - 然后我们就发现可以通过 $O(n)$ 的处理得出每一部分得权重, 然后贪心最大化
- 注意这里有一个小细节: $r \geq l$, 所以我们在处理的时候需要注意处理 l 得同时处理 r
- 这里还有个实现层面的优化: 你可能会发现如果对于每个 r 都要扫一边 l 我们前面做的降维全都白费了, 我们通过同时维护 $ans1$ 和 $ans2$ (两个需要最大化的贪心模块)让他在一边扫的时候可以同时更新同时过。这也是一个重要的trick

```
for (int i = 1; i <= n; i++)
{
    if (chaxun2[i] - chaxun2[zuobian] > ans2)
    {
        ans2 = chaxun2[i] - chaxun2[zuobian];
    }
}
```

```

    }
    if (chaxun2[i] < ans1)//这里的更新顺序是因为我们的优化已经让i=1-1了，所以顺序
不能错
    {
        ans1 = chaxun2[i];//边处理边更新线性扫过去是非常常见的优化方法
        zuobian = i;
    }
}

cout << qzh[n] + ans2 << '\n';

```

```

void solve()
{
    int n;
    cin >> n;
    vi nums(n + 1);
    vi qzh(n + 1);
    vi chaxun1(n + 1);
    vi chaxun2(n + 1);
    for (int i = 1; i <= n; i++)
    {
        int d;
        cin >> d;
        nums[i] = d;
    }
    for (int i = 1; i <= n; i++)
    {
        qzh[i] = qzh[i - 1] + nums[i];
        chaxun1[i] = (i * i) + i - qzh[i];
        chaxun2[i] = (i * i) + i - qzh[i];
    }
    int ans1 = 0;
    int zuobian = 0;
    int youbian = 0;

    int ans2 = 0;

    for (int i = 1; i <= n; i++)
    {
        if (chaxun2[i] - chaxun2[zuobian] > ans2)
        {
            ans2 = chaxun2[i] - chaxun2[zuobian];
        }
        if (chaxun2[i] < ans1)
        {
            ans1 = chaxun2[i];
            zuobian = i;
        }
    }
}

```



```
cout << qzh[n] + ans2 << '\n';
}
```

C. Monopati

- [link](#)
- **思路** 题意转换：对于每个合法的路径，必然存在某一个合法的转换列 i 让他从上行过度到下行。因为要求出 l, r 的区间所以我们需要得上行从前往后的最大值和最小值，下行从后往前的最小值和最大值。✓ 对于每个 i ，路径存在的充要条件是： $l \leq L_i$ 且 $r \geq R_i$ 。（其中 L_i 是该路径上的最小值， R_i 是该路径上的最大值）。→最终目标：统计 (l, r) 的数量，使得 存在至少一个 i 满足上述条件。即求 n 个约束区域（矩形）的 并集 覆盖的整数点数。如果对每个点 i 进行分析太难了，我们可以考虑进一步转化问题：

要求合法的 l, r 对数:固定每一个 l ，统计合法的 r 的数量。原理：**固定一个变量，用数据结构（堆/线段树）维护另一个变量的可行域**。更新方程是： $sum += 2 * n + prdq.top()[0]$;

容器选择：理想的实现应该做到:过滤过期：随着 l 增大，剔除那些 $L_i < l$ 的约束（因为它们不再能覆盖当前的 l ）。查询最优：在剩下的合法约束中，找到 R_i 最小的那个（ R 越小，合法的 r 越多）。

易被忽视的细节：对于每个 r 的约束是：我们找到的和他配对的 l 必须小于 r 在的数对的 l 所以在堆里的排序规则是： r 越小越好， l 越大越好。这样子我们筛掉的点对既不会漏掉 r 也不会漏掉 l

- `priority_queue<array<int, 2>> prdq;`默认按照`prdq[0][0]`大小排序
- **AC代码**(优先队列法)

```
void solve()
{
    int n;
    cin >> n;
    vi mpu(n);
    vi mpd(n);
    vi mxu(n);
    vi mnu(n);
    vi mxd(n);
    vi mnd(n);
    for (int i = 0; i < n; i++)
    {
        cin >> mpu[i];
        mpu[i]--;
        if (!i)
        {
            mxu[i] = mpu[i];
            mnu[i] = mpu[i];
        }
        else
        {
            mxu[i] = max(mxu[i - 1], mpu[i]);
            mnu[i] = min(mnu[i - 1], mpu[i]);
        }
    }
}
```

```

for (int i = 0; i < n; i++)
{
    cin >> mpd[i];
    mpd[i]--;
}
for (int i = n - 1; i >= 0; i--)
{
    if (i == n - 1)
    {
        mxd[i] = mpd[i];
        mnd[i] = mpd[i];
    }
    else
    {
        mxd[i] = max(mxd[i + 1], mpd[i]);
        mnd[i] = min(mnd[i + 1], mpd[i]);
    }
}
priority_queue<array<int, 2>> prdq;
for (int i = 0; i < n; i++)
{
    prdq.push({-max(mxd[i], mxu[i]), min(mnd[i], mnu[i])});
}
ll sum = 0;
for (int i = 0; i < 2 * n; ++i)
{
    while (!prdq.empty() && prdq.top()[1] < i)
    {
        prdq.pop();
    }
    if (!prdq.empty())
    {
        sum += 2 * n + prdq.top()[0];
    }
}
cout << sum << '\n';
}

```

P2887 [USACO07NOV] Sunscreen (收录意义是请和C. Monopati对比学习)

• 题目

题目描述 有 C 头奶牛进行日光浴，第 i 头奶牛需要 $minSPF[i]$ 到 $maxSPF[i]$ 单位强度之间的阳光。每头奶牛在日光浴前必须涂防晒霜，防晒霜有 L 种，涂上第 i 种之后，身体接收到的阳光强度就会稳定为 $SPF[i]$ ，第 i 种防晒霜有 $cover[i]$ 瓶。求最多可以满足多少头奶牛进行日光浴

• 思路

- Monopati也是用到了这个思路。把l固定然后遍历r。但是贪心是要证明无后效性的。就是你这个牛子过了之后你能保证他后面没有可能匹配到别人。最适合的就是拿右值来排序
- 我本来写的区间大小，但是这样子就和我通过统一左值来做的方法矛盾

- 如果一方有‘容忍范围’ (min-max) , 可以:固定一边 (排序) , 扫描另一边, 并用优先队列维护‘最紧急/最容易过期’的选项。”

- **AC代码**

```
struct spf
{
    int low, hei;
};
struct fss
{
    int sp, fg;
};

void solve()
{
    int c, l;
    cin >> c >> l;
    vector<spf> cows(c);
    rep(i, 0, c - 1)
    {
        cin >> cows[i].low >> cows[i].hei;
    }
    vector<fss> choi(l);
    rep(i, 0, l - 1)
    {
        cin >> choi[i].sp >> choi[i].fg;
    }

    sort(all(choi), [](fss a, fss b)
        {
            if(a.sp!=b.sp)return a.sp<b.sp;
            else return a.fg>b.fg; });
    sort(all(cows), [](spf a, spf b)
        {
            if(a.low!=b.low)return a.low<b.low;
            else return a.hei>b.hei; });
    priority_queue<int, vector<int>, greater<int>> > dui;

    int ans=0;
    int niuniu=0;

    for(int i=0;i<l;i++)
    {
        while(niuniu<c&& cows[niuniu].low<=choi[i].sp)
        {
            dui.push(cows[niuniu].hei);
            niuniu++;
        }
        while(!dui.empty()&& dui.top()<choi[i].sp)
        {
            dui.pop();
        }
    }
}
```

```

    }
    while(choi[i].fg>0&&!dui.empty())
    {
        ans++;
        choi[i].fg--;
        dui.pop();
    }
}
cout<<ans;
}

```

P2859 [USACO06FEB] Stall Reservations S(收录意义是学会读题以及请和Sunscreen一起学习)

- 这道题可以和上面三道题以及后悔贪心那道题一起看
- 个人认为这道题并不贪心，只是一个堆的使用典例。我认为是Monopati的前戏
- 这里主要在证明为什么用这样的贪心策略:先排序左端点，然后将右端点如堆。可以保证对于每一次的运算不会漏掉情况。因为可能漏掉的情况一定同时满足以下两个条件:左端点比当前位置更前，右端点比当前位置更前。可以简单的发现根据这样的排列不存在例外插入情况。
- 这里依旧贴出大根堆代码，用来多多评鉴。
- 这里有一些小细节，因为最后要输出每个牛在那个栅栏，这个细节调了挺久的

```

sort(all(nums), [](hsh a, hsh b)
    {
        if(a.st!=b.st) return a.st<b.st;
        else return a.et<b.et; });

priority_queue<pii, vector<pii>, greater<pii>> dui;
vector<hsh> ans(n);
int weilan = 1;
for (int i = 0; i < n; i++)
{ // 对于第i头牛
    auto it = nums[i];
    int now=0;
    if (dui.empty())
    {
        dui.push({it.et, weilan});
        ans[i].et = weilan;
        ans[i].id = it.id;

        continue;
    }
    auto ljl = dui.top();

    if (it.st <= ljl.first)
    {
        weilan++;
        dui.push({it.et, weilan});

        now=weilan;
    }
}

```

```

        else
        {
            dui.pop();
            dui.push({it.et, ljl.second});
            now=ljl.second;
        }
        ans[i].et=now;
        ans[i].id=it.id;

    }
    cout << dui.size() << '\n';
    sort(all(ans), [](hsh a, hsh b)
        { return a.id < b.id; });
    for (auto jjj : ans)
    {
        cout << jjj.et << '\n';
    }

```

贪心

- 如上P2887 [USACO07NOV] Sunscreen就是典型的贪心，重点是如何去通过我们固定左端点的大胆尝试去实现

P2949 [USACO09OPEN] Work Scheduling G (全局贪心)

- **我一开始的错误思路**
 - 人人都能想得到去走时间，然后将最紧急的任务先做了然后再紧急的任务里面贪心
 - 最大的问题是:你的贪心是占用时间的，是有后效性的，万一后面有价值更高的任务他就不是期望值了
 - DP? 时间跨度巨大: $D_i \leq 10^9$ 。任务数量 (N) 较大: $N \leq 10^5$ 。离散化时间之后依然n方
- **正确思路以及为什么:** 以后你在做题时，如果满足以下特征，请立刻想到反悔贪心:

选择带有顺序性（或者可以排序）。

当前的选择会影响后续的资源（比如占用了时间、金钱）。

我们在乎的是“数量”或者“总价值”。

如果发生冲突，我们可以通过“撤销”之前的某个劣质选择，来接纳当前的优质选择，且这种交换一定是不亏的（甚至更赚）

- **AC代码**

```

struct hsh
{
    int d, p;
};

void solve()

```

```

{
    int n;
    cin >> n;
    vector<hsh> nums(n);
    rep(i, 0, n - 1)
    {
        cin >> nums[i].d >> nums[i].p;
    }
    ll ans = 0;
    sort(all(nums), [](hsh a, hsh b)
        { return a.d < b.d; });
    // int dpin = 0;
    // int xpin = 0;
    int time1 = 0;
    priority_queue<int,vi,greater<int>> dui;
    for(auto ljl:nums) //记住这个反悔的思想和实现方式
    {
        if(time1+1<=ljl.d)
        {
            dui.emplace(ljl.p);
            time1+=1;
        }
        else
        {
            if(!dui.empty())
            {
                auto ljl2=dui.top();
                dui.pop();
                if(ljl.p>ljl2)
                {
                    dui.emplace(ljl.p);
                }
                else dui.emplace(ljl2);
            }
        }
    }
    while(!dui.empty())
    {
        auto jj=dui.top();
        dui.pop();
        ans+=jj;
    }
    cout << ans << '\n';
}

```

构造

C. Cyclic Merging(贪心)

- 题目

- You are given n non-negative integers $a_1, a_2, \dots, a_n$ arranged on a ring. For each $1 \leq i < n$, a_i and a_{i+1} are adjacent; a_1 and a_n are adjacent. You need to perform the following operation **exactly** $n - 1$ times: Choose any pair of adjacent elements on the ring, let their values be x and y , and merge them into a single element of value $\max(x, y)$ with cost $\max(x, y)$. Note that this operation will decrease the size of the ring by 1 and update the adjacent relationships accordingly. Please calculate the minimum total cost to merge the ring into one element.

给你一个排列在环上的非负整数

$$n$$

。对于每个

$$1 \leq i < n$$

,

$$a_i$$

和

$$a_{i+1}$$

相邻;

$$a_1$$

和

$$a_n$$

相邻。以下操作需要精确

$$n - 1$$

次：— 在环上选择任意一对相邻的元素，设它们的值为

$$x$$

和

$$y$$

，并将它们合并成一个值为

$$\max(x, y)$$

，代价为

$$\max(x, y)$$

的元素。注意，此操作将使环的大小减少

$$1$$

\$, 并相应地更新相邻关系。请计算将圆环合并为一个元件的最低总成本。

• 错误想法

- 双指针：双指针/滑动窗口通常用于处理固定不变的序列，寻找满足某种条件的连续子段（比如和、最值等）。而这道题首先是环形，其次是他会破坏掉数字的结构
- DP？数据范围 $2e5$ 秒了肯定不是

$2e5$ 我们最好想一些数据结构或者贪心（某种意义上的排列）来优化这一切（ $\log n$ 或者 n ，这很常见）

• 思考路径(看完整清晰的请往下看，这是我个人的一个思路整理)

- 当我拆解数据 1 1 4 5 1 4 1 的时候我总是会算错，这个样例我研究了20分钟我才研究出来他的正确合成方法:而这道题的正确做法就藏在我的试错里面:只是我忽略了:当我从小合并到大(其实这是猜猜看的结果)他最优
- 贪心最大的难度就是让我相信：他是贪心
- 相信他是贪心最主要的点在：对于每个数他的命运岔路口只有当左右两边的数字都大于他的时候。可以简单想到，通过更改(合成)左右两个数来改变消掉我们这个数的代价绝对不是最优，最优解**存在且仅存在于**他与隔壁的一个数字合体或着他与隔壁两个数字中更小的数字合体
- 这里我们可以发现从最小的数字开始合并收益一定最大(如果从一个不是最大也不是最小的数字开始合并，他的消失他就可能会影响更小的数字合成代价变大。因为他消失了更小的数字只能和更大的数字合成了)

• 细节难点：

- 实现方法上需要学习的:模拟环形(模拟链表)以及左右更新
- 大根堆的处理:要认清楚:大根堆的最用只是帮你快速定位到最小的元素是谁

• AC代码

```
struct node
{
    int idx, num;
};

struct cmp
{
    bool operator()(const node &a, const node &b)
    {
        return a.num > b.num; // 小根堆逻辑
    }
};

void solve()
{
    int n;
    cin >> n;
    vi nums(n);
    vi left(n);
    vi right(n);
```



```

left[0] = n - 1;
right[n - 1] = 0;
priority_queue<node, vector<node>, cmp> dl;
rep(i, 0, n - 1)
{
    cin >> nums[i];
    if (i != 0)
    {
        left[i] = i - 1;
    }
    if (i != n - 1)
    {
        right[i] = i + 1;
    }
    dl.push({i, nums[i]});
}

int cz = n - 1;
ll ans=0;
while (cz && !dl.empty())
{
    auto hsh = dl.top();
    dl.pop();
    int idl = left[hsh.idx];
    int idr = right[hsh.idx];

    if(nums[idl]>nums[idr])ans+=nums[idr];
    if(nums[idl]<=nums[idr])ans+=nums[idl];
    // cerr<<ans<<'\n';
    left[idr]=idl;
    right[idl]=idr;
    cz--;
}
cout<<ans<<'\n';
}

```

你以为到这里就结束了吗？并非 诚然这个算法很直观，但是却不是最简单，没有找到这道题最本质的一点 **我们容易发现**: merge 取的是 max，这意味着**邻居只会越变越大**。对于一个当前的最小值来说，“现在”永远是它能遇到的最好时机。任何推迟操作都可能导致它身边的邻居变大，从而增加它最终被消除的代价。

这样子我们就能获得时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 的算法:

```

void solve()
{
    int n,first,now=0;
    ll ans = 0;
    cin >> n;
    cin >> first;
    int last = first;
    int mx=first;
    rep(i, 2, n)

```

```

{
    cin >> now;
    mx = max(mx, now);
    ans += max(last, now);
    last = now;
}
ans+=max(first,now);
cout<<ans-mx<<'\n';
}

```

- 后话:我们该如何证明贪心是对的

- 如果数据范围是 $N = 2 \times 10^5$ ，而且不像线段树/图论，那只能是贪心或者数学推导。
- 贪心规则，检查有没有后效性，更优？

复杂度优化

C. Range Operation

- 从 n 方缩减成 $O(n)$,我们通过分离lr与计算贡献的方式抽象题目的问法

LIS

- 题目：给个序列nums，求他的最长上升子序列
- $O(n^2)$ 做法:

$$dp[i] = \max(dp[j]) + 1 \quad \text{其中 } 0 \leq j < i, a[i] > a[j]$$

- $O(n \log n)$ 做法:

```

int LIS_nlogn(vector<int>& a) {
    if (a.empty()) return 0;

    // low[i] 存的是长度为 i+1 的子序列的最小结尾
    // 注意：为了方便，low 的下标从 0 开始，所以 low[len-1] 是长度为 len 的结尾
    vector<int> low;

    for (int x : a) {
        // Case 1: 如果 low 为空，或者 x 比 low 最后一个元素大
        if (low.empty() || x > low.back()) {
            low.push_back(x);
        }
        // Case 2: 否则，在 low 里找到第一个 >= x 的位置，替换它
        else {
            // lower_bound 返回迭代器，指向第一个 >= x 的元素
            auto it = lower_bound(low.begin(), low.end(), x);
            *it = x; // 贪心：让该长度的结尾变得更小
        }
    }

    return low.size();
}

```

• 优化思考路径

- 这是一种优化贪心，将dp对象转换成能达到的长度，每一位都代表着如果要达到这个长度我最小要有多大，精辟在更新。相当于转换了dp的两个对象：
- ==最长能有多长->如果要达到这个长度需要最小什么条件==

• ==应用==:

- **Dilworth定理**：最少的下降序列个数就等于整个序列最长上升子序列的长度
- LCS解法->将一个数组的每一个数字跟他的位置映射在一起，相当于开了个桶

- **注意**：求下降序列需要`auto it= upper_bound(all(hei),hsh,greater<int>());`。普通的公式只能算：排列好后的不下降数列

错题整理

记录错题，可能只是神秘UB，还有伟大的边界处理

复杂度优化

P2058 [NOIP 2016 普及组] 海港

- **错点**：TLE
- 错题记录：

一开始我想用quene来模拟时间维度上的滑动，结果发现quene不支持遍历。于是用了deque 第一次本地WA：我想用set记录每艘船上面的游客种类，然后和他的时间一起emplace进去队列，ans+=每一个时间点的种类。属于是读错题了。第二次提交TLE：我想那就开一个map来存所有数据，然后遍历放进set里面进行去重...数据范围K总和3e5，n为2e5随爆炸 后来乱交了两罚，因为想不通

- 打开题解：用桶来装数字，当这个桶里面的数字是第一次被装到1，他就多一个种类，反之少一个种类。需要将出队判断移到后面。实现线性处理ans，实在精妙

//然后自己手搓了了，AC

• AC代码

```
struct ship
{
    int t;
    int gj;
};

void solve()
{
    int n;
    cin >> n;
    deque<ship> bc;
    vi nums(1e5 + 2);
```

```

int ans = 0;
rep(i, 0, n - 1)
{
    int t, k;
    cin >> t >> k;
    // 这里要写一个出队判断
    rep(j, 0, k - 1)
    {
        int d;
        cin >> d;
        nums[d]++;
        if (nums[d] == 1)
            ans++;
        bc.push_back({t, d});
    }
    while (!bc.empty() && bc.front().t <= t - 86400)
    {
        auto hsh = bc.front();
        nums[hsh.gj]--;
        if (nums[hsh.gj] == 0)
            ans--;
        bc.pop_front();
    }
    cout << ans << '\n';
}
}

```

题意理解&&容器选择

I题是个签到题

- 错误原因：**在处理这个条件或者通过人数是所有题目前三多的题（也就是最多有两个题目通过人数严格比它多）叫做签到题。的时候第一次用的set，本来想的是找到第三大的数字然后直接比较，获得set的rbegin() (set容器从小到大排)。结果发现：bro，他说是最多两个题目。。。谢谢你的脑抽：贡献了13发罚时 == 以后觉得有问题先考虑是不是读错题了 ==

边界处理

P2004 领地选择

- 分析**
 - 题目是非常简单而且典型的滑动窗口二维前缀和。最大的问题出在你滑动窗口的时候的推导式：详情见P2280 HNOI2003 激光炸弹
- 出错的推导点** $int\ hsh = qzh[i + c - 1][j + c - 1] + qzh[i - 1][j - 1] - qzh[i + c - 1][j - 1] - qzh[i - 1][j + c - 1];$
- 首先注意减去边界格的时候要-1往里面一格（因为要包含到ij），然后是因为是C*C的数组所以要往里面移动一格（请你以后先列式子算一下）

子段乘积（简单）

- 题号: C
- 链接: [题目链接](#)
- 算法类型: 滑动窗口 (双指针)
- AC 代码:

```
void solve()
{
    int n, k;
    cin >> n >> k;
    vi nums(n + 1);
    rep(i, 1, n)
    {
        cin >> nums[i];
    }

    int left = 1;
    int right = 1;
    int ans = 0;
    int sz = 1;
    int mod = 998244353;
    while (right <= n)
    {
        while (right <= n && left + k > right)
        {
            if (nums[right] == 0)
            {
                left = right + 1;
                right = left;
                sz = 1;
                continue;
            }
            sz = (sz * nums[right]) % mod;
            right++;
        }
        if (right > n || left > n - k)
            break;

        // if (right - left == k)这是保险起见之举，事实上没有必要
        ans = max(ans, sz);

        sz = (sz * inverse_fermat(nums[left], mod)) % mod;
        left++;
    }
    cout << ans;
}
```

- 注意事项:
 - 1-Based存数方式lr初始在1

- 记得滑动窗口的while，不要惯性思维认为答案更新/大小比较都在while的末尾。while只是一个路径，前面是快指针移动，后面是慢指针移动。中间夹着你的数字更新
- 适用于固定区间长度的数据处理
- 维护r的界限和nums[r]的更新逻辑是难点 [相似的滑动窗口](#)

赛后补题

网络赛也是赛

11月2日华南理工大学秋季新生赛（第一场）

- **名次** 总榜36，校内榜5
- **存在问题** 签到题出太慢（可恶的devcpp），看不懂题目（泥工怎么出题不给题目名词解释！！太不规范了！！），D题读题失误，错写dp！！（有环图！）

因数分解

- **题号**: G
- **链接**: [题目链接](#)
- **算法类型**: 二分查找
- **错误原因**:
 - 看不懂题目在讲啥

有 n 堆石子排成一行，从左到右依次编号为 1 到 n ，第 i 堆有 (a_i) 个石子。trilliverse 和宇暖微霄两人轮流进行游戏，trilliverse 先手。每回合，当前玩家必须选择**最左边的石子数大于 0 的石子堆**（即下标最小的非空石子堆），石子数量为 (a_i) ，并从中取走 (k) 个石子，其中 (k) 必须满足以下条件之一：

$$k = a_i$$

$[\forall \text{forall } i, a_i \equiv 0 \pmod{a_i - k}]$

!!对于每一个 i , a_i 都能被 $a_i - k$ 整除，即 $a_i - k$ 是 a_i 的除数。

- **AC 代码**:

```
void solve()
{
    int n;
    cin >> n;
    bool by = (n % 2 == 1);
    vi nums(n + 1);
    rep(i, 1, n)
    {
        int d;
        cin >> nums[i];
    }
    if (n == 1)
    {
        cout << "CandidateMaster" << '\n';
        return;
    }
}
```

```

    }
    rep(i, 1, n)
    {
        if ( i % 2 == 0)
        {
            if (nums[i] != 1)
            {
                cout << "LegendaryGrandmaster" << '\n';
                return;
            }
        }
        else if ( i % 2 == 1)
        {
            if (nums[i] != 1)
            {
                cout << "CandidateMaster" << '\n';
                return;
            }
        }
    }
    if (by)
        cout << "CandidateMaster" << '\n';
    else
        cout << "LegendaryGrandmaster" << '\n';
}

```

- **解析:**
 - **关键点**想到对于每个人来说取石子的最优解是==当石子大于1的时候取 a_i-1 ==
 - 可以想到，先手拿到的是奇数位上的控制权，后手拿到偶数上的控制权。一旦某人拿到大于1的数字，就可以通过取 k 个 ($a_i-k=1$) 来拿到后续的控制权。从而把后面所有的情况压缩为1
 - 博弈疑问？为什么可以压缩？万一后面的人可以通过在后来拿到主动权来改变先后手优势吗？答：注意到如果要改变先后手优势，你就得走到别人的位置上（先手走偶数，后手走奇数）如果后面还有主动权，我大可以跳过前面的所有选项来获得我必有的控制权。
- **改进思路:**
 - 思维题，一般这种简单博弈都不要想的太难。一般某个人能拿到绝对主动权，看绝对主动权给谁最重要

爆破鸭科夫

- **题号:** L
- **链接:** [题目链接](#)
- **算法类型:** 二分查找
- **错误原因:**
 - 看不懂题目在讲啥
- **AC 代码:**

- **注意事项:**
 - 注意二分查找的左右边界初始化。
 - 确保 check 函数逻辑正确。
- **改进思路:**
 - 考虑严格 $\text{cnt} == k$ 的情况，调整 check 函数

水群

- **题号:** D
- **链接:** [题目链接](#)
- **算法类型:** Disjkstr最短路
- **错误原因:**
 - 看成DP了!! 有向无环有向无环有向无环!
 - 迪克算法还在追我
 - 最短路问题
- **AC 代码:**

```
void solve()
{
    【输入】
    vector<vector<pair<ll, int>>> mp(n + 1);
    for (int i = n-1; i >= 1; i --)
    {
        mp[i].push_back({x, i + 1});
        mp[i + 1].push_back({x, i});
    }
    rep(i, 0, m - 1)
    {
        int a, b;
        cin >> a >> b;
        mp[a].push_back({y, b});
        // mp[b].push_back({y, a});
    }
    【迪克算法】
    cout<<dist[1]<<'\n';
}
```

- **思路:**
 - 没什么好说的，迪克算法模板，我都懒得贴出代码，详情请见[简单 Dijkstra 模板题](#)

代号N

- **题号:** E
- **链接:** [题目链接](#)
- **算法类型:** 带权无向图[相似题目学习](#)
- **错误原因:**
 - 看不懂一些[基础名词](#)

精简题干: 定义一棵树: 共 n 个节点: 一个度数为3的节点, 若干个度数为2的节点, 3个度数为1的节点. 给出 $n-1$ 条边, 给出两个端点以及边权, 可以操作 k 次将某条边边权变成0. 求根节点到叶节点的度数最大值的最小值.

• AC 代码:

```
struct edge
{
    int to, w;
};
int cmp(const vi &sums)
{
    if(sums[0]>sums[1])
    {
        if(sums[0]>sums[2])
        {
            return 0;
        }
        else return 2;
    }
    else
    {
        if(sums[1]>sums[2])
        {
            return 1;
        }
        else return 2;
    }
}
void solve()
{
    int n, k;
    cin >> n >> k;
    vector<vector<edge>> mp(n + 1);
    vi nums(n + 1);
    for (int i = 0; i < n-1; i++)
    {
        int u, v, w;
        cin >> u >> v >> w;
        mp[u].push_back({v, w});
        mp[v].push_back({u, w});
        nums[u]++;
        nums[v]++;
    }
    auto it = find(all(nums), 3);
    int sdot = (it - nums.begin());
    vector<vector<int>> chain(3);
    vi sums(3);
    vector<priority_queue<int>> pq(3); // <--- 改动: 为每条链维
    护大根堆
    for (int i = 0; i < 3; i++)
    {
        queue<pii> pos;
```

```

        pos.emplace(mp[sdot][i].to, sdot);
        chain[i].push_back(mp[sdot][i].w);
        sums[i] += mp[sdot][i].w;
        pq[i].push(mp[sdot][i].w); // <--- 改动: 把边权加入堆

        while (!pos.empty())
        {
            auto [now, par] = pos.front();
            pos.pop();

            for (auto &nextdot : mp[now])
            {
                if (nextdot.to == par)
                    continue;
                else
                {
                    chain[i].push_back(nextdot.w);
                    sums[i] += nextdot.w;
                    pq[i].push(nextdot.w); // <--- 改动: 把遍历到的边权加入堆

                    pos.emplace(nextdot.to, now);
                }
            }
        }
    }
    while(k--)
    {
        int idx = cmp(sums);
        if (pq[idx].empty()) break; // 防止空链
        int mx = pq[idx].top();
        pq[idx].pop(); // O(log len)
        sums[idx] -= mx;
        // 下面的两行可以删掉, 保留只为保持原结构
        // sums[cmp(sums)] -= *max_element(all(chain[cmp(sums)]));
        // chain[cmp(sums)].erase(max_element(all(chain[cmp(sums)])));
    }
    cout << sums[cmp(sums)];
}

```

• 思路剖析:

- 非常简单的大数据和和的最大值的最小值.解法一:sort之后逐个pop_back()(注意vector的pop是最后一位,所以要从大到小排序).解法二:维护最大根堆priority_queue<int>,n次操作复杂度nlogn.
- 事实上解法一是优解,但是为了学习这个有意思的容器这道题用的做法是最大根堆
- 这道题的难点是读懂题...然后是建带权无向图...依旧模板.
- **记得**带权无向图两个端点都pushback一次
- 然后就是用bfs带着上一个节点和下一个节点广度优先探索(这个还能用dfs解法,下文附上,作为图论路径转移学习,请严肃学习114514次)

大部分图论用BFS做,在面临剪枝需求/需要回溯/所有方案的时候用DFS做

• 细节注意:

- 二维拓展数组记得这样开 `vector<vector<edge>> mp(n + 1);` 不会爆空间
- Cpp17不支持结构体取地址,就这么写 `auto [now, par] = pos.front();`
- 记得k次操作不一定用玩一定要提前出循环 `if (pq[idx].empty()) break;` 以免UB

• 新结构

- `priority_queue`: 自动维护堆的“插入+弹出最大/最小”工具, “贪心/最短路/Top-K/滑动窗口最大值” 都能靠它快速实现
- 注意优先队列没办法删除除了堆顶以外的元素,所以**注意**`if (d > dist[u]) continue;`

声明代码	含义
<code>priority_queue<int> pq;</code>	大根堆 (默认, 队头最大)
<code>priority_queue<int, vector<int>, greater<int>> pq;</code>	小根堆 (队头最小)
<code>priority_queue<pair<int,int>> pq;</code>	pair 大根堆 (first 越大越前)
<code>priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;</code>	pair 小根堆 (first 越小越前)

操作	代码	说明	复杂度
插入	<code>pq.push(x);</code>	插入元素 x	O(log n)
取顶	<code>pq.top();</code>	只读 队头	O(1)
删除顶	<code>pq.pop();</code>	删除队头	O(log n)
判空	<code>pq.empty();</code>	是否为空	O(1)
大小	<code>pq.size();</code>	元素个数	O(1)

爆零什么的千万不要啊

- **题号:** A (最难, 防AK题)
- **链接:** [题目链接](#)
- **算法类型:** 二分查找
- **记录原因:**
 - 非常有意思的处理方法, 已严肃学习
- **AC 代码:**

- **注意事项:**
 - 注意二分查找的左右边界初始化。
 - 确保 `check` 函数逻辑正确。
- **改进思路:**

- 考虑严格 $\text{cnt} == k$ 的情况，调整 check 函数

11月2日晚牛客挑战赛

小红的区间构造

- 算法：前缀和
- 思路&错误原因
 - 这道题想要你构造区间，最坏情况是所有点都是一个区间。题目给出来的条件就非常前缀和，直接差分处理然后一个一个insert左右端点就完事儿了
- 注意事项
 - 注意数据范围**提前退出**以免mle（致敬我调了一个晚上的点）
 - 注意学习左右端点的处理方法：因为是差分处理所以两个数组大小一定一样
- AC代码

```
void solve()
{
    int n, m;
    cin >> n >> m;
    vi nums(n + 2);
    vi cf(n + 2);
    int sum=0;
    for (int i = 1; i <= n; i++)
    {
        cin >> nums[i];
        sum += nums[i];
        if (nums[i] > m)
        {
            cout << "-1" << '\n';
            return;
        }
        cf[i] = nums[i] - nums[i - 1];
    }
    if(sum < m){cout << "-1" << '\n';return;}
    cf[n + 1] = nums[n + 1] - nums[n];
    vi left;
    vi right;
    for (int i = 1; i <= n + 1; i++)
    {
        if (cf[i] > 0)
            for (int j = 0; j < cf[i]; j++)
                left.push_back(i);
        if (cf[i] < 0)
            for (int j = 0; j < abs(cf[i]); j++)
                right.push_back(i - 1);
    }
    int k = left.size();
    int gap = m - left.size();
    if (left.size() > m)
    {
```

```

        cout << "-1" << '\n';
        return;
    }
    ll cnt=0;
    for (int j = 0; j < k; j++)
    {
        cnt+= right[j] - left[j] + 1;
    }
    if(cnt<m)
    {
        cout << "-1" << '\n';
        return;
    }

    for (int j = 0; j < k; j++)
    {
        if(gap==0)
            cout<<left[j]<<' '<<right[j]<<'\n';

        if (right[j] - left[j] < gap&& gap!=0)
        {
            for (int i = left[j]; i <=right[j]; i++)
            {
                cout<<i<<' '<<i<<'\n';
            }
            gap--(right[j] - left[j]);
            continue;
        }
        if (right[j] - left[j] >= gap && gap!=0)
        {
            for (int i = left[j] ; i < left[j]+gap; i++)
            {
                cout<<i<<' '<<i<<'\n';
            }

            cout<<left[j]+gap<<' '<<right[j]<<'\n';
            gap=0;
            continue;
        }
    }
}

```

2025年11月5日晚上牛客基础组

子段异或和

- 题号: D
- 链接: [题目链接](#)
- 算法类型: 异或, 前缀和
- AC 代码:

```

void solve()
{
    int n;
    cin >> n;
    vi nums(n + 1);
    nums[0] = 0;
    vi cf(n + 1);
    map<int, int> bg;
    int ans = 0;
    bg[0]++;
    for (int i = 1; i <= n; i++)
    {
        cin >> nums[i];
        cf[i] = cf[i - 1] ^ nums[i];
        int key = cf[i] ^ 0; // a^b=0 -> a^0=b
        if (bg.find(key) != bg.end())
        {
            ans += bg[key];
        }
        bg[cf[i]]++;
    }

    cout << ans;
}

```

• 异或前缀和思路:

- 异或的“逆运算”就是它自己。
- 所以通过求异或前缀和来实现对 $[l, r]$ 区间的异或和的 $O(1)$ 查找——>就等于 $cf[r] \oplus cf[l-1]$ 。
- 异或的一些小妙招:
 - 寻找出现次数偶数/奇数次数数列里面出现奇数/偶数次数的数
 - N堆石子，每次可以取任意多个至少一个。异或和 $\neq 0$ 先手必败

• 改进技巧: 哈希表:

- $O(N)$ 优化 (使用哈希表): 我们用一个哈希表 countMap 存储历史上的前缀和 $cf[j]$ 出现过的次数 (复杂度 $O(\log N)$)
- 能 $O(1)$ 查询的字段和都可以采用

“前缀和+哈希表”是处理“子数组/子段”的小妙招

子段乘积 (简单)

- 题号: C
- 链接: [题目链接](#)
- 算法类型: 滑动窗口 (双指针)
- AC 代码:

```

void solve()
{
    int n, k;
    cin >> n >> k;
    vi nums(n + 1);
    rep(i, 1, n)
    {
        cin >> nums[i];
    }

    int left = 1;
    int right = 1;
    int ans = 0;
    int sz = 1;
    int mod = 998244353;
    while (right <= n)
    {
        while (right <= n && left + k > right)
        {
            if (nums[right] == 0)
            {
                left = right + 1;
                right = left;
                sz = 1;
                continue;
            }
            sz = (sz * nums[right]) % mod;
            right++;
        }
        if (right > n || left > n - k)
            break;

        // if (right - left == k)这是保险起见之举，事实上没有必要
        ans = max(ans, sz);

        sz = (sz * inverse_fermat(nums[left], mod)) % mod;
        left++;
    }
    cout << ans;
}

```

• **注意事项:**

- 1-Based存数方式l初始在1
- 记得滑动窗口的while，不要惯性思维认为答案更新/大小比较都在while的末尾。while只是一个路径，前面是快指针移动，后面是慢指针移动。中间夹着你的数字更新
- 适用于固定区间长度的数据处理
- 维护r的界限和nums[r]的更新逻辑是难点

C

- **记录原因**：介绍可以进行 $O(\log n)$ 复杂度查询的容器：set, map, multiset
- **思路**：可以轻易的发现，我们需要贪心：分成两块，如果有奖励（可以不断刷新剑的伤害）我们先做，后面再开没有奖励的怪物。该死，我明明已经会了，结果写的时候脑子糊糊的忘记了。。。对于有奖励的组，我们需要遍历怪物（来达到每个怪物只能打一次的效果）。因为怪物是可以sort的，sort的规则是血量从小到大。因为我们知道再有奖励的组别里面剑的攻击力只会增加不会减少。。
 - 我们优化的难点在如何降低复杂度：我们要找到能打败这个怪物条件下攻击力最小的剑，传统的维护最小值是 $O(n)$ ，我猪脑过载想到的是sort加二分 $O(\log n)$ 。这个时候就出现我们伟大的基于红黑树的快速查找容器set, map, multiset。
 -

==**优化场景**== 动态维护有序集合， $O(\log n)$ 查找/插入/删除。 可以实现二分查找lower_bound和upper_bound set用来去重 map用来根据键值对快速查找。

- **AC 代码**:

```
struct mon
{
    int b, c;
};

void solve()
{
    int n, m;
    cin >> n >> m;
    vi a(n);
    vector<mon> mnst(m);
    rep(i, 0, n - 1)
    {
        cin >> a[i];
    }
    rep(i, 0, m - 1)
    {
        cin >> mnst[i].b;
    }
    rep(i, 0, m - 1)
    {
        cin >> mnst[i].c;
    }
    vector<mon> youjl;
    vector<mon> meijl;
    rep(i, 0, m - 1)
    {
        if (mnst[i].c == 0)
        {
            meijl.push_back(mnst[i]);
        }
        else
            youjl.push_back(mnst[i]);
    }
}
```



```
int ms = meijl.size();
int ys = youjl.size();
int ans = 0;
sort(all(youjl), [](mon aa, mon bb)
     { if(aa.b!=bb.b) return aa.b<bb.b;
       else return aa.c > bb.c;});
sort(all(meijl), [](mon aa, mon bb)
     { return aa.b<bb.b; });
multiset<int> hsh;
rep(i, 0, n - 1)
{
    hsh.emplace(a[i]);
}

for (int j = 0; j < ys; j++)
{
    if (!hsh.empty())
    {
        auto cnm = hsh.lower_bound(youjl[j].b);
        if (cnm != hsh.end())
        {
            int z1 = *cnm;
            hsh.emplace(max(youjl[j].c, z1));
            ans++;
            hsh.erase(cnm);
        }
    }
}

for (int j = 0; j < ms; j++)
{
    if (!hsh.empty())
    {
        auto cnm = hsh.lower_bound(meijl[j].b);
        if (cnm != hsh.end())
        {
            ans++;
            hsh.erase(cnm);
        }
    }
    if (hsh.empty())
    {
        break;
    }
}

cout << ans << '\n';
}
```

广州大学2024新生赛VP

在ACM里面打ACM (map的使用: 模拟: 数据处理)

-AC代码

```
struct hsh
{
    bool accepted = false;
    int wrangans = 0;
    int time = 0;
};
void solve()
{
    int n;
    cin >> n;
    map<string, map<char, hsh>> ljl;
    map<string, int> sumtime;
    map<string, int> sumac;
    string ans;
    for (int i = 0; i < n; i++)
    {
        string n;
        char th;
        bool ac;
        int tme;
        cin >> n >> th >> ac >> tme;
        ans = n;
        if (ljl[n][th].accepted)
            continue;
        if (ac)
        {
            ljl[n][th].time += tme;
            ljl[n][th].time += ljl[n][th].wrangans * 20;
            ljl[n][th].accepted=true;
            sumac[n]++;
            sumtime[n] += ljl[n][th].time;
        }
        else
        {
            ljl[n][th].wrangans++;
        }
    }

    for (auto it : ljl)
    {
        if (sumac[it.first] > sumac[ans])
        {
            ans = it.first;
        }
        if (sumac[it.first] == sumac[ans])
        {
```

```

        if (sumtime[it.first] < sumtime[ans])
        {
            ans = it.first;
        }
    }
}
cout<<ans<<' '<<sumac[ans]<<' '<<sumtime[ans];
}

```

逆=1+剪

放在这里是为了告诉你：正确分析复杂度是多么的重要。。。以及数学好真的很重要 以及有时候大脑真的不要把题目想的太难，可以根据过题人数考虑是否需要考虑唐氏做法 这道题纯纯预处理一下就行了 //可能我通宵了，大脑垃圾太多了，竟然放过了一道这么唐氏且简单还珍贵的题目

来自数院的小楠喜欢研究数学，他给定了一个数字 (n)，然后提出了 (q) 个问题，每个问题给出两个整数 (k) 和 (r)，现在他想知道，在他的每个问题中，他给出的数字 (n) 是否满足**不少于 (k) 个不同的正整数** ($c_1, c_2, c_3, \dots, c_k$) 能够使得： $[n \bmod c_i = r, \quad i = 1, 2, 3, \dots, k.]$ 其中 **mod** 代表求余，例如：

($5 \bmod 2 = 1$)

($8 \bmod 2 = 0$) 第一行输入两个整数 $n(1 \leq n \leq 1 \times 10^6)$, $q(1 \leq q \leq 1 \times 10^4)$ ，分别代表小楠给出的这个数字 n 和提出的问题数量 q 。接下来 q 行，每行输入两个整数 $k(1 \leq k \leq 1 \times 10^6)$, $r(0 \leq r < n)$ ，代表 n 是否满足有不少于 k 个不同的正整数 $c_1, c_2, \dots, c_k$ 使得 $n \bmod c_i = r$ 。输出共 q 行，如果 n 能够满足第 i 个问题，则输出 "YES"，否则输出 "NO"。（输出不含双引号，且必须严格大写）

```

void solve()
{
    int n, q;
    cin >> n >> q;
    vi nums(n + 1);
    for (int i = 1; i <= n; i++)
    {
        nums[n % i]++;
    }
    for (int i = 1; i <= q; i++)
    {
        int k, r;
        cin >> k >> r;
        if (k <= nums[r])
            cout << "YES" << '\n';
        else
            cout << "NO" << '\n';
    }
}

```

1.图论名词

名词	无歧义定义（竞赛语境）
简单图	不含 自环 且任意两顶点之间 至多一条边 的图。
多重图	允许同一对顶点之间存在 多条边 （平行边）。
自环	起点与终点为同一顶点的边，记作 $((u,u))$ 。
邻接	若存在边连接顶点 (u) 与 (v) ，则称 (u) 与 (v) 相邻 。
度数	顶点 (u) 所连接的 边数 ；无向图中记作 $(\deg(u))$ ，有向图分为 出度 与 入度 。
路径	顶点序列 $(v_0, v_1, \dots, v_k)$ ，相邻顶点间均有边；长度为边数 (k) 。
简单路径	路径中 无重复顶点 。
环	起点与终点相同的路径（闭合路径）。
简单环	除起点与终点外 无重复顶点 的环。
连通	无向图中任意两顶点之间 存在路径 。
强连通	有向图中任意两顶点 (u,v) 均满足 $(u \rightarrow v)$ 与 $(v \rightarrow u)$ 均有路径。
连通分量	无向图的 极大连通子图 。
强连通分量 (SCC)	有向图的 极大强连通子图 。
树	无向连通无环图 ； (n) 个顶点恰有 $(n-1)$ 条边。
生成树	连通无向图的 无环连通子图 ，包含所有顶点。
叶子节点	度数为 1 的节点（树中常见）。
根节点	树中指定的起点节点（有根树）。
子树	以某节点为根及其所有后代构成的树。
边权	每条边附加的数值（整数/实数），常表示距离、费用等。
加权图	边带有权值的图。
最短路径	路径上 边权之和最小 的路径。
负权边	权值为负数的边。
负环	环上权值和为负的环。
二分图	顶点可分为两个互斥集合 (A,B) ，且 所有边均连接 (A) 与 (B) 。
完全二分图	二分图中 (A) 中每个点与 (B) 中每个点均有边。
匹配	二分图中 边集无公共顶点 的子集。
最大匹配	包含 边数最多 的匹配。
完美匹配	匹配覆盖 所有顶点 。
DAG	有向无环图 （Directed Acyclic Graph）。

名词	无歧义定义（竞赛语境）
拓扑序	DAG 的顶点线性序列，满足：若 $(u \rightarrow v)$ ，则 (u) 在 (v) 前。
割点	删除后 连通分量数增加 的顶点。
桥	删除后 连通分量数增加 的边。
点双连通	删除任意单个顶点后仍连通（无割点）。
边双连通	删除任意单条边后仍连通（无桥）。
欧拉路径	经过每条边恰好一次 的路径。
欧拉回路	闭合 的欧拉路径。
哈密顿路径	经过每个顶点恰好一次 的路径。
哈密顿回路	闭合 的哈密顿路径。
最小生成树 (MST)	连通加权无向图中 边权和最小 的生成树。
最大流	从 源点 到 汇点 的 最大流量 （容量限制）。
最小割	容量和最小的 割集 ，等于最大流（最大流最小割定理）。
残余网络	网络流中记录 剩余容量 的辅助图。
增广路径	残余网络中从源到汇的路径，用于提升流量。
LCA	树上两节点 深度最大 的公共祖先（最近公共祖先）。
树直径	树上 最长简单路径 的长度。
差分约束	一组形如 $(x_i - x_j \leq c_k)$ 的不等式，可转化为最短路求解。
费用流	每单位流量有费用，求 最小费用 的 最大流 。

2.计算几何常用